Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

SISTEMA DE AUTORIZACIÓN DE FACTURAS ELECTRÓNICAS (DTE) Y GESTIÓN DE RTU

202300379– Mónica Sofia Boche Figueroa

Resumen

El presente informe técnico describe la arquitectura, diseño e implementación de una solución web para el procesamiento masivo, validación y autorización de Documentos Tributarios Electrónicos (DTE) y la actualización del Registro Tributario Unificado (RTU). Desarrollado en la plataforma .NET utilizando C#, el sistema adopta un modelo de arquitectura desacoplada Cliente-Servidor compuesto por una API RESTful en el Backend y una aplicación web interactiva en el Frontend con ASP.NET Core Razor Pages. La persistencia y manipulación de datos se efectúan en memoria mediante estructuras orientadas a objetos, garantizando tiempos de respuesta mínimos en el procesamiento de transacciones. El sistema incorpora un panel de visualización estadística enriquecido con gráficos interactivos desarrollados en Chart.js y una interfaz adaptativa con estilo visual personalizado.

Palabras clave

1. API REST
2. C# .NET
3. Facturación Electrónica (DTE)
4. Razor Pages
5. Arquitectura Cliente-Servidor

Abstract

This technical report presents the architecture, design, and implementation of a web solution engineered for the bulk processing, validation, and authorization of Electronic Tax Documents (DTE) and Taxpayer Registry (RTU) management. Developed on the .NET framework using C#, the application employs a decoupled Client-Server architecture comprising a RESTful Backend API and an interactive ASP.NET Core Razor Pages Frontend. Data persistence and manipulation are handled entirely in-memory using object-oriented structures, guaranteeing minimal latency during high-volume transaction processing. The system features an analytics dashboard enhanced with interactive charts powered by Chart.js and a responsive user interface built with a customized visual palette.

Keywords

1. REST API
2. C# .NET
3. Electronic Invoicing (DTE)
4. Razor Pages
5. Client-Server Architecture

Introducción

En el contexto actual del comercio electrónico y la transformación digital de la administración tributaria, la automatización en el procesamiento de documentos fiscales exige sistemas capaces de validar, autorizar y registrar transacciones financieras de alto volumen con máxima eficiencia.

Este proyecto presenta la implementación de una plataforma integral desarrollada en C# .NET para la recepción, validación de reglas de negocio, cálculo de impuestos (IVA 12%) y emisión de autorizaciones de solicitudes de facturación electrónica. La arquitectura desacoplada garantiza la separación de responsabilidades entre la lógica de negocio consumida mediante servicios web HTTP y la capa de presentación consumida por el usuario final.

1. Arquitectura del Sistema

El sistema se estructura bajo una Arquitectura Cliente-Servidor Desacoplada, permitiendo que el Frontend (capa de presentación) interactúe con el Backend (servicios web) exclusivamente mediante peticiones HTTP asíncronas con formato de intercambio de datos JSON.

2. Descripción del Backend (API REST Core)

El backend expone un conjunto de Endpoints diseñados para recibir estructuras JSON de entrada, validar su consistencia y procesar las autorizaciones.

2.1 Módulos y Flujos de Trabajo

1. Gestión de RTU (POST /api/procesar-rtu):

- Registra nuevos contribuyentes o actualiza la información existente (NIT, Nombre, Dirección, Estado).
- Almacena los registros en estructuras en memoria de acceso rápido.

2. Procesamiento de Facturas (POST /api/procesar-facturas):

- Validación de Contribuyentes: Verifica la existencia y validez del NIT del emisor y receptor en el RTU.
- **Cálculo Tributario:** Calcula el Débito y Crédito Fiscal aplicando la tasa del 12% de IVA sobre el valor base del monto total.
- Correlativos y Autorización: Asigna un número de autorización único y correlativo a las facturas aprobadas.
- Gestión de Errores: En caso de fallas (NIT no registrado, fechas inválidas, montos incoherentes), clasifica la solicitud como rechazada y adjunta el código de error correspondiente.

3. Consulta de Reportes (GET /api/reportes):

- Retorna el consolidado de métricas globales: total de solicitudes, facturas autorizadas, facturas rechazadas y montos acumulados de IVA.

4. Reinicio del Sistema (POST /api/reset):

- Restablece la memoria interna del servidor, limpiando el registro de contribuyentes, facturas y contadores correlativos.

4. Descripción del WebFrontend (Razor Pages)

El frontend proporciona una interfaz intuitiva, moderna y accesible orientada al usuario administrador.

4.1. Vistas Principales

- Carga de Datos (CargarDatos.cshtml)**:** Posee áreas de texto interactivas para el envío masivo de fragmentos JSON para la actualización de RTU y el lote de solicitudes de facturación, mostrando en tiempo real el resultado en JSON devuelto por la API.
- Dashboard y Reportes (Reportes.cshtml): Presenta un panel de control con tarjetas de métricas primarias, una gráfica radial/dona generada asíncronamente con **Chart.js** para

la proporción de facturas aprobadas/rechazadas, y un desglose tabular detallado con opción de exportación a documento PDF.

4.2. Estilo Visual y Paleta de Colores

Para brindar una experiencia visual profesional e identificable, el diseño del frontend implementa una paleta de tonos orgánicos integrada mediante variables CSS en site.css:

- Verde Oliva (#576d2c): Utilizado en encabezados, barra de navegación y botones principales de acción.
- Verde Musgo (#b9b26c): Aplicado en acentos de tarjetas, bordes e indicadores secundarios.
- Terracota (#cc6333): Utilizado para estados de alerta, métricas de error y botones de reinicio/cancelación.
- Coral / Salmón (#e78169): Aplicado en destacados de tablas, etiquetas gráficas de estado rechazado y elementos cebra.

5. Modelado Orientado a Objetos (Modelos de Datos)

5.1. Modelos Principales

Clase Contribuyente

- NIT: String (Identificador único)
- Nombre: String
- Direccion: String
- Estado: String ("ACTIVO", "INACTIVO")

Clase Factura

- NumeroFactura: Int / String
- NitEmisor: String
- NitReceptor: String
- Fecha: DateTime
- MontoTotal: Decimal
- MontoIva: Decimal
- Estado: String ("APROBADA", "RECHAZADA")
- CodigoAutorizacion: String
- MensajeError: String

Clase ResumenReporte

- TotalProcesadas: Int
- TotalAprobadas: Int
- TotalRechazadas: Int
- MontoTotalIva: Decimal

6. Guía de Ejecución y Despliegue

Para realizar la evaluación del proyecto, se requiere la ejecución simultánea de ambos proyectos mediante dos instancias de consola independiente:

1. Servidor Backend

```
cd Backend
dotnet run
Escuchando en: http://localhost:5234
```

2. Cliente Frontend

```
cd WebFrontend
dotnet run
Escuchando en: http://localhost:5248
```



Figura 1. Diagrama de Clases

Fuente: elaboración propia.

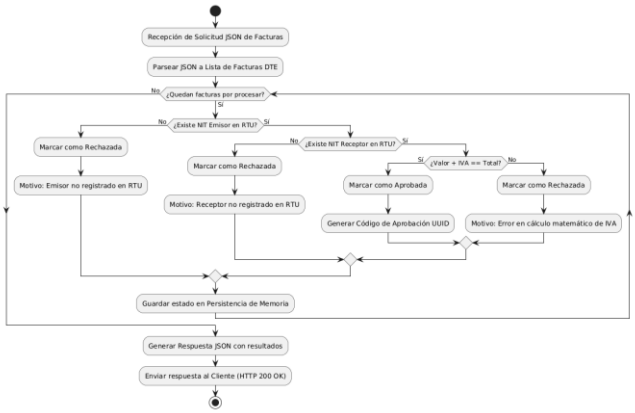


Figura 2. Diagrama de Actividades

Fuente: elaboración propia.



Figura 3. Diagrama de Despliegue

Fuente: elaboración propia.

Conclusiones

- La adopción de una arquitectura desacoplada basada en servicios REST facilitó el desarrollo modular del sistema, permitiendo modificar la lógica tributaria en el backend sin afectar la interfaz visual del usuario.
- La gestión y persistencia temporal en memoria redujo los tiempos de procesamiento de lotes masivos de datos a milisegundos, ofreciendo respuestas inmediatas en las consultas estadísticas del frontend.
- La integración de componentes dinámicos de visualización de datos (Chart.js) junto con una paleta de colores personalizada enriqueció la presentación del informe ejecutivo, mejorando la interpretabilidad de las métricas clave del sistema.

Referencias bibliográficas

- Microsoft Learn. (2024). *Tutorial: Crear una API Web con ASP.NET Core*. Microsoft.
- Microsoft Learn. (2024). *Introducción a Razor Pages en ASP.NET Core*. Microsoft.
- Chart.js Documentation. (2023). *Chart.js: Simple yet flexible JavaScript charting for designers & developers*.
- W3C. (2023). *JSON Data Interchange Format Specifications*.